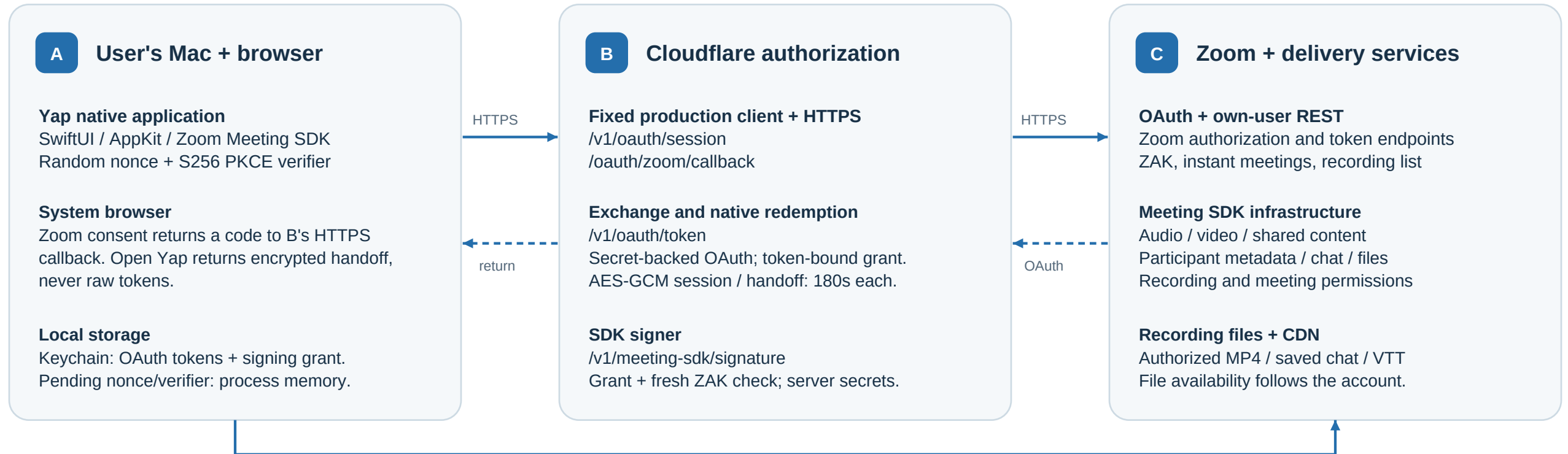


Managed Zoom authorization

Fixed HTTPS callback, encrypted native return, and direct meeting/media paths



Direct A-to-C paths: browser consent, own-user REST, SDK media, chat/files and recording downloads

Meeting content, recording passcodes and Google Calendar data bypass the authorization service.

Six exchanges, one native sign-in

- 1 Mac creates the session**
POST nonce + verifier to B. B seals them with audience, fixed callback and expiry; returns a Zoom URL with PKCE challenge.
- 2 Browser authorizes at Zoom**
C returns code + sealed state to B's exact HTTPS callback. B validates state and exchanges the code using its production secret.
- 3 Browser returns an encrypted handoff**
B seals tokens, grant, nonce and verifier hash. Open Yap uses yap://oauth/zoom; plaintext code, verifier and tokens stay out of that URL.
- 4 Initiating Mac redeems over HTTPS**
Mac consumes pending nonce once and POSTs handoff + original verifier to B. B validates proof and returns the remaining token lifetime.
- 5 Mac saves and refreshes credentials**
After own-user ZAK validation, tokens + grant go to Keychain. Later refresh uses B and its confidential production OAuth credentials.
- 6 Service authorizes the Meeting SDK**
Mac sends access token + bound grant. B verifies the grant and fresh ZAK access at Zoom, then signs the short-lived SDK JWT.

Trust boundaries and deployment configuration

Same source design; separate production and development identities and secrets

Configured environments

P Production review candidate

meeting-auth.mgrinich.workers.dev
OAuth + SDK ID: UHoml3aIQpy86gZeijfpQ
HTTPS callback: /oauth/zoom/callback
Native app uses this production origin and ID.

D Development environment

meeting-auth-development.mgrinich.workers.dev
OAuth + SDK ID: ZA20iVuUSmSKMGVmx7tDyA
HTTPS callback: /oauth/zoom/callback
Separate secrets and rate-limit namespaces.

Worker routes

GET /connect - native integration entry
POST /v1/oauth/session - sealed sign-in state
GET /oauth/zoom/callback - fixed Zoom return
POST /v1/oauth/token - handoff redemption / refresh
POST /v1/meeting-sdk/signature - validated SDK JWT
GET /health - health only, not OAuth acceptance

Secrets and API scope

Server secrets: production OAuth / SDK secret and an independent signing-grant secret. OAuth and SDK IDs must match before the same secret is used. Each environment needs its own secrets.

Zoom scopes: user:read:zak; meeting:write:meeting; cloud_recording:read:list_user_recordings.
Only the consenting user's data; no admin/master scope.

Memory, encryption and replay boundary

No persistent server user/token/session database. Plaintext codes, verifiers, OAuth tokens, ZAKs and grants are processed in request memory. Session and handoff are separately authenticated AES-GCM envelopes, each expiring within 180 seconds. Browser history may retain callback URLs and ciphertext; expiry does not erase it. Desktop nonce is consumed once. The stateless broker has no redeemed-handoff ledger: replay before expiry is possible with the original verifier. Token expiry is never extended by redemption.

Local data and direct providers

Keychain holds tokens and signing grants. Calendar cache, playback storage, saved chat/attachments, group photos and imported camera backgrounds have separate local retention. Google OAuth + read-only Calendar API remain direct Mac-to-Google. Media redirects drop Zoom authorization before a non-Zoom CDN target.

Bounded service processing

Fixed upstream URLs; no credential redirects; 15s upstream timeout; bounded request/response bodies. Limits: 120 requests/minute per hashed source IP; 10 signatures/minute per token hash. Constant error events only; invocation logs and traces off. Provider infrastructure metadata is separate.

Compatibility: released 0.1.6 retains legacy public-client loopback; personal mode keeps direct Zoom OAuth + local SDK signing. This diagram is source architecture, not deployment, live-test or Zoom-approval evidence.